

NCA-Desafío-CSI1

Diario de trabajo

1º DAM (Desarrollo de aplicaciones multiplataforma)

Autores:

Alejandro Mosquera.

Daniel Fernandez.

Gloria Colmenarez.

La Salle Santo Ángel

2025/26

Asignaturas:

Inglés

Programación

Bases de Datos

Lenguaje de Marcas

Sistemas Informáticos

Entornos de desarrollo

Distribución de Roles y Responsabilidades.....	7
1. Daniel: Desarrollo y Sistemas con Máquinas Virtuales (MV).....	7
2. Alejandro: Gestión de Datos y Entornos.....	7
3. Gloria: Diseño de User Experience (UX), Sistemas informáticos y pruebas de integración.....	8
Diagrama Canvas:.....	9
1. Programación.....	11
3. Entornos de Desarrollo.....	12
4. Sistemas Informáticos.....	12
5. Diseño y Maquetación Web: Lenguaje de Marcas.....	13
6. Inglés.....	13
Definición de tecnologías implementadas y su entorno:.....	14
Fase IV: Materialización.....	15

Abstract

This project presents a comprehensive solution for managing a digital bookstore, unifying software development in Java and MySQL with advanced network infrastructure design. The system enables efficient management of inventory, sales processes, and book loans. The architecture is supported by a network design created in Cisco Packet Tracer using subnets, optimizing security and data flow. Furthermore, the software stands out for its use of regular expressions to validate data and an intuitive graphical interface, complemented by a website to maximize its commercial reach.

The objective or purpose of this project is to develop a professional system to manage the stock, sales, and loans of a digital bookstore. We aim to create a solution that is secure through data validation and has a wide reach thanks to its future integration with a website, thereby facilitating internal organization and customer service.

Keyword: Bookstore, management system, data validation.

Introducción:

El presente proyecto consiste en el desarrollo de una aplicación útil con el lenguaje Java diseñada para la gestión integral de una librería digital. El sistema está estructurado bajo una arquitectura de Programación Orientada a Objetos (POO), lo que permite la realización de un CRUD, ante lo dicho, es importante señalar que un CRUD es un acrónimo de las cuatro operaciones fundamentales que cualquier sistema de gestión de datos debe realizar. En Java, cuando hablamos de la "parte gráfica", nos referimos a que estas operaciones no se hacen escribiendo comandos en una consola negra, sino a través de ventanas, botones y tablas. En este sentido cada palabra significa lo siguiente: C(create), R (Read), Update), D (delete), estas definiciones se utilizan en interfaz gráfica como Swing o JAVAFX. Donde se usan JTextField, JButton, JTable, entre otros elementos que garantizan una conexión eficiente entre entidades relacionadas al momento de utilizar la aplicación, como lo son los clientes, empleados, libros, rentas, pagos y el programa en general.

En el mismo orden de ideas, se realiza un trabajo sobre una arquitectura funcional, dividida en módulos, donde el módulo cliente está diseñado como una interfaz transaccional dinámica que gestiona el registro de usuarios y la lógica de negocio aplicada al alquiler y compra de libros, incluyendo el cálculo automatizado de impuestos. Por otro lado, el panel de administración actúa como el núcleo de gestión del sistema, donde el personal autorizado y empleados, mediante una validación de credenciales, supervisan la integridad del inventario y el mantenimiento de los registros de usuarios, asegurando un control total sobre las operaciones de la biblioteca.

Finalmente, el trabajo sigue una metodología de ingeniería profesional de Software profesional, donde se planifica con diagramas y se gestiona los datos y tiempos equitativamente, así como, los flujos y rutas de datos. En correlación, se implementará el GitHub como sistema de control y versiones, lo que asegura la integración de modificaciones y evolución del proyecto en conjunto, protegiendo la documentación, el código y posibles eventualidades presentadas

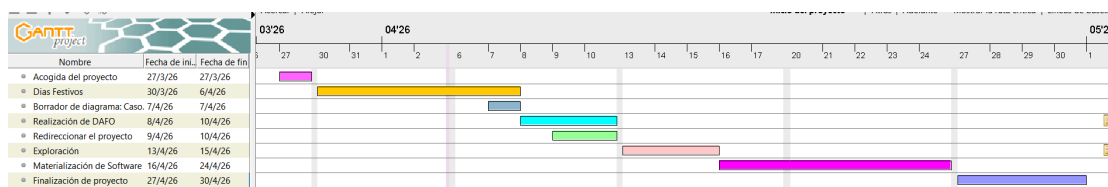
Fase I: Idear

En esta etapa inicial del proyecto, el equipo se enfocó en buscar soluciones creativas y accesibles para estructurar de manera óptima el sistema de gestión de la biblioteca. A través de sesiones de lluvia de ideas y borradores realizados a mano, que definan el proceso del trabajo y cómo estas fases se interconectan.

El objetivo principal en esta fase fue planificar una plataforma integral que no solo fuera eficiente a nivel técnico en el manejo de bases de datos y programación, sino que también resultara intuitiva, moderna y completamente adaptada a dispositivos móviles para el usuario final, además del desarrollo planificado de tareas y la disposición económica, para que sea un proyecto escalable y factible, tanto para el equipo como para el usuario final.

Diagrama de Gantt:

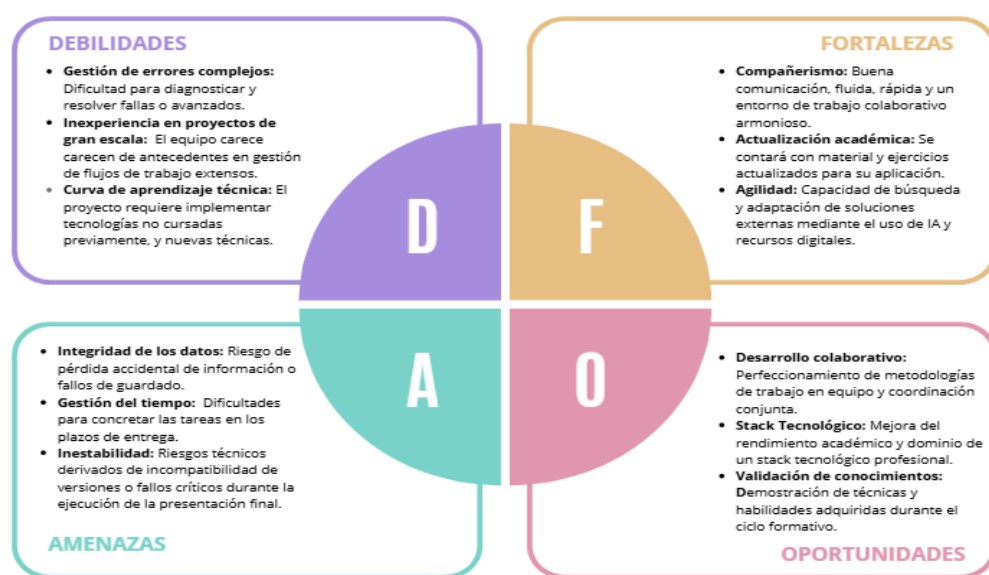
Este diagrama organiza las fechas y los plazos para desarrollar y entregar el proyecto. Es una herramienta viva que se irá actualizando con las tareas específicas de cada integrante del equipo. La planificación se basa en dos pilares: la buena comunicación del grupo y el control de los tiempos día a día. Además, se han incluido notas detalladas en las fases más importantes para explicar exactamente qué se necesita hacer en cada paso. De esta manera, todo el equipo sabe qué se espera de ellos y cómo evitar retrasos.



Matriz DAFO:

Para la realización de este proyecto, es importante realizar un análisis DAFO, el cual tiene como objetivo evaluar la situación actual del equipo de trabajo frente al desarrollo del mismo, mediante la integración tecnológica a gran escala. A través de la Matriz DAFO, se identifican los factores internos y externos que condicionarán el éxito del despliegue, permitiendo una planificación lo más clara posible en el cronograma de actividades.

El diagnóstico ha revelado al grupo la unión interna y capacidad de respuestas rápidas, respaldado por una formación académica actualizada. No obstante, el equipo reconoce desafíos significativos relacionados con la curva de aprendizaje en nuevas tecnologías y la gestión de riesgos técnicos operativos, es decir, existe información desconocida para el grupo lo que permite a los miembros actualizar los conocimientos que ya tiene y profundizar con técnicas importantes para el proyecto. Este análisis no solo expone las vulnerabilidades del proyecto, sino que también, establece bases para aprovechar las oportunidades profesional, y encontrar posibles amenazas que presenta el equipo .



Distribución de Roles y Responsabilidades

A continuación, se detalla la asignación de tareas basada en las competencias técnicas y perfiles de cada integrante:

1. Daniel: Desarrollo y Sistemas con Máquinas Virtuales (MV)

- **Áreas:** Programación de interfaces y Sistemas con MV.
- **Perfil:** Se caracteriza por su capacidad de investigación y autoaprendizaje constante. Posee un enfoque orientado a la experimentación con código nuevo para lograr interfaces funcionales y estéticas.
- **Competencias técnicas:** Soltura previa en entornos Windows y disposición para la gestión de sistemas Linux.
- **Gestión de riesgo:** Deberá monitorizar los tiempos de ejecución para evitar que la fase de experimentación comprometa los plazos de entrega.

2. Alejandro: Gestión de Datos y Entornos

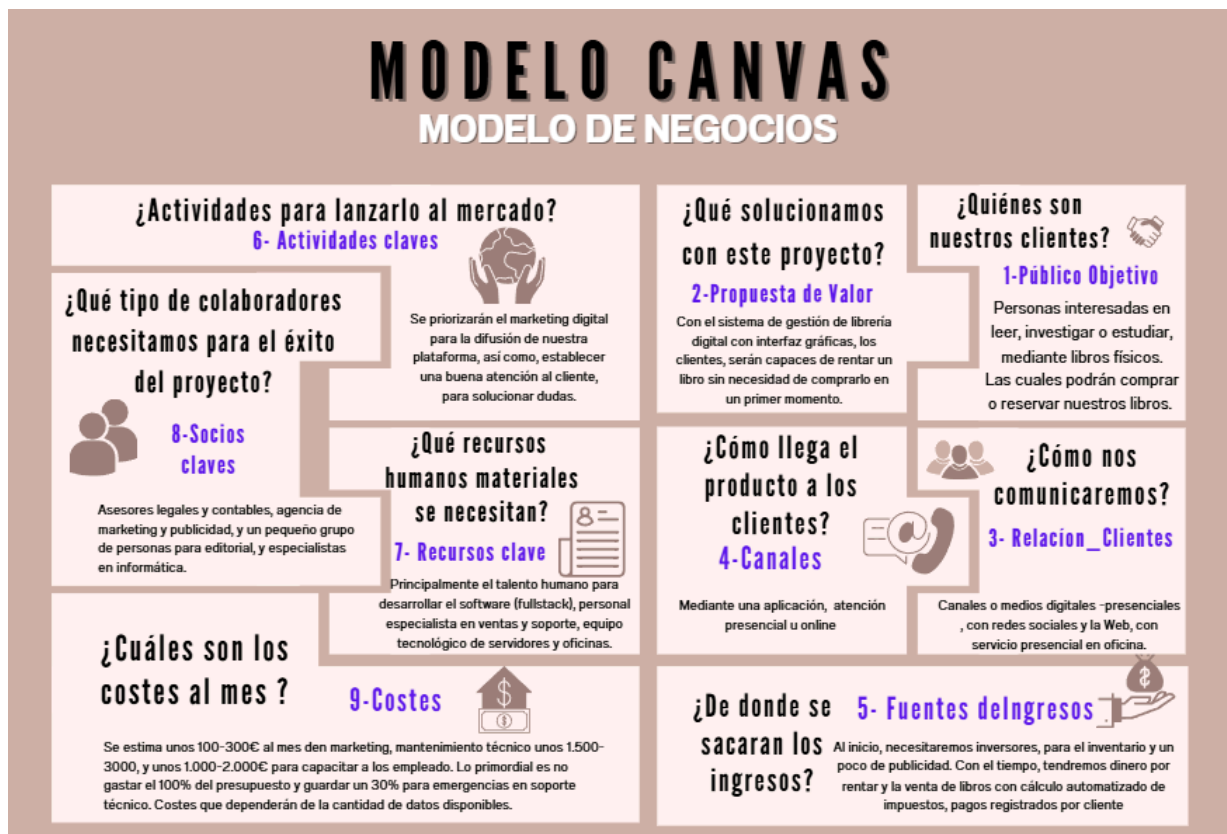
- **Áreas:** Bases de datos, Lenguaje de marcas y Entornos de desarrollo.
- **Perfil:** Amplia visión para diversos problemas y gran enfoque para probar nuevos puntos de vista a la hora de resolver los obstáculos que se le presenten, domina Full-Stack.
- **Competencias técnicas:** Dominio de lenguajes de marcas y Entorno de Desarrollo.
- **Gestión de riesgo:** Ya que maneja 3 asignaturas y quiere participar en la creación y diseño de la App, el mayor desafío será la organización.

3. Gloria: Diseño de User Experience (UX), Sistemas informáticos y pruebas de integración.

- **Áreas:** Diseño de interfaz gráfica con el lenguaje Java y Sistemas informáticos con instaladores y registros.

- **Perfil:** Aporta el enfoque de experiencia de usuario, asegurando que la aplicación sea intuitiva y visualmente coherente.
- **Competencias técnicas:** Dominio en programación y sus especialización en la parte gráfica, lo que garantiza coherencia entre el front-end y el back-end
- **Gestión de riesgo:** El éxito de su labor dependerá de una coordinación constante con el área de programación para alinear el diseño visual con la implementación del código.

Diagrama Canvas:



Fuente: Colmenarez(2026).

Lógica de Negocio y restricciones de Integridad:

En el presente proyecto, para garantizar la fiabilidad del sistema, se han implementado restricciones tanto en la base de datos como en la lógica

de Java. Por ejemplo, el sistema impide el registro de usuarios menores de 12 años, además de validar que no existan correos duplicados y asegura que las fechas de devolución de libros sean posteriores a la fecha de entrega, manteniendo así la coherencia temporal de los préstamos y el orden en la librería.

Fase II: Exploración de plataformas y tecnologías

El sistema se desarrollará bajo un entorno de arquitectura cliente-servidor local utilizando Máquinas virtuales, mediante JDBC (Java Database Connectivity), donde la IP es creada o asignada por la VM, que a su vez, el JDBC usa la IP como si fuera una dirección en un GPS, y poder transmitir los datos correctamente entre sí. Como por ejemplo, transmitir la información de la base de datos al programa que interactúa con el cliente.

En el mismo orden de ideas, se ha seleccionado Java como lenguaje de programación por su capacidad de gestión orientada a objetos y MySQL como motor de base de datos relacional. La recolección de metadatos de libros se apoya en fuentes abiertas como Open Library API para garantizar la veracidad de los registros de ISBN y títulos.

Fuentes de información y herramientas utilizadas:

Para garantizar la veracidad y calidad de los datos en nuestro sistema, se ha recolectado información de fuentes oficiales y en lugar de utilizar datos ficticios. La recopilación de información bibliográfica se obtiene de manera automatizada mediante la integración de la API Google Books, la cual permite localizar libros por ISBN, título, autor o palabras clave. Lo que facilita la consulta de catálogos de instituciones de referencia como la Biblioteca Nacional para estandarizar géneros y categorías.

En cuanto a la estructura del sistema, los medios de recolección se han basado en requisitos internos, mediante sesiones de trabajo e investigación para definir funciones necesarias y críticas del sistema, además de un análisis de plataformas líderes como Kindle y Goodreads para asegurar que nuestro modelo de datos cumpla con los estándares actuales del mercado.

Recursos digitales:

Considerando que este trabajo se realiza con la finalidad de diseñar un sistema informático para administrar una biblioteca. Por lo que, para su desarrollo y la resolución de los retos que se presentaron, el equipo utilizó una serie de recursos digitales de apoyo, como :

Sistemas de Inteligencia Artificial: Herramientas informáticas como Gemini, ChatGPT o Antigravity se emplearon de manera controlada y exclusivamente como un asistente de consulta para resolver dudas o buscar ideas de enfoque.

Comunidades Virtuales: Se realizaron consultas en plataformas web de soporte técnico y colaboración entre programadores, destacando los casos de Stack Overflow y los almacenes de código de GitHub.

Aplicación de herramientas en las asignaturas

1. Programación

En este bloque nos enfocamos en escribir las instrucciones y el código que hacen funcionar la aplicación de la biblioteca.

- **Tecnología empleada:** El diseño del software se realizó utilizando el lenguaje **Java**, y para escribir las líneas de código de forma ordenada nos servimos de la plataforma de desarrollo **Apache NetBeans**.
- **Fuentes de consulta:** Para resolver las dificultades del código acudimos al foro especializado *Stack Overflow*, revisamos las actividades prácticas guiadas por el profesor en las clases previas y consultamos la documentación que ofrece la compañía *Oracle*.

● 2. Bases de Datos

Esta sección se encarga de estructurar el espacio donde se almacenarán de forma permanente todos los registros del sistema, como los datos de los libros disponibles y las fichas de los usuarios.

- **Tecnología empleada:** Toda la gestión y el diseño de las tablas de datos se llevó a cabo mediante el entorno gráfico **MySQL**.
- **Fuentes de consulta:** El punto de partida consistió en plasmar nuestras ideas en papel mediante esquemas y diagramas hechos a mano. Una vez definida la lógica, trasladamos esa estructura al programa informático.

3. Entornos de Desarrollo

Aquí incluimos todas las aplicaciones secundarias que nos ayudan a planificar el proyecto, organizar las tareas y escribir de forma más cómoda.

- **Tecnología empleada:** El editor principal para revisar los archivos fue **Visual Studio Code**. Para crear los esquemas visuales que explican cómo interactúa el usuario con el programa, utilizamos la herramienta digital **Draw.io**, apoyándonos también en anotaciones en un cuaderno tradicional.
- **Fuentes de consulta:** El avance de este bloque se logró repasando los ejercicios del curso, revisando ejemplos públicos en la plataforma *GitHub* y consultando con la *Inteligencia Artificial* para aclarar conceptos complejos.

4. Sistemas Informáticos

Este apartado consistió en preparar un entorno seguro en la computadora para hacer pruebas de funcionamiento y diseñar la manera en que el usuario final instalará la aplicación.

- **Tecnología empleada:** Se utilizó el software **VirtualBox** para simular sistemas operativos independientes (como Linux) dentro de nuestros propios ordenadores. También trabajamos en el empaquetado del archivo que servirá para instalar el programa de forma automática, sin olvidar el uso del Packet Tracer para simular la distribución del programa por las redes.
- **Fuentes de consulta:** Nos guiamos por los contenidos y las prácticas realizadas durante el segundo trimestre, especialmente los comandos de Linux compartidos en el aula, además de buscar soporte en foros de internet y software de IA para estructurar el instalador.

5. Diseño y Maquetación Web: Lenguaje de Marcas

Consiste en la creación de la página web oficial de la biblioteca, que sirve como interfaz visual para que las personas accedan al servicio por internet.

- **Tecnología empleada:** La web se construyó utilizando los estándares de internet: **HTML** para definir el contenido y **CSS** para darle estilo y colores. Con el fin de que la página se adapte automáticamente al tamaño de las pantallas de móviles y tabletas, implementamos la tecnología de diseño **Bootstrap**. Todo el desarrollo se hizo en **Visual Studio Code**.
- **Fuentes de consulta:** Reutilizamos la estructura de los trabajos web que elaboramos durante el año académico y ampliamos los conocimientos investigando sobre las reglas de *Bootstrap* y el almacenamiento de datos en archivos *XML*.

6. Inglés

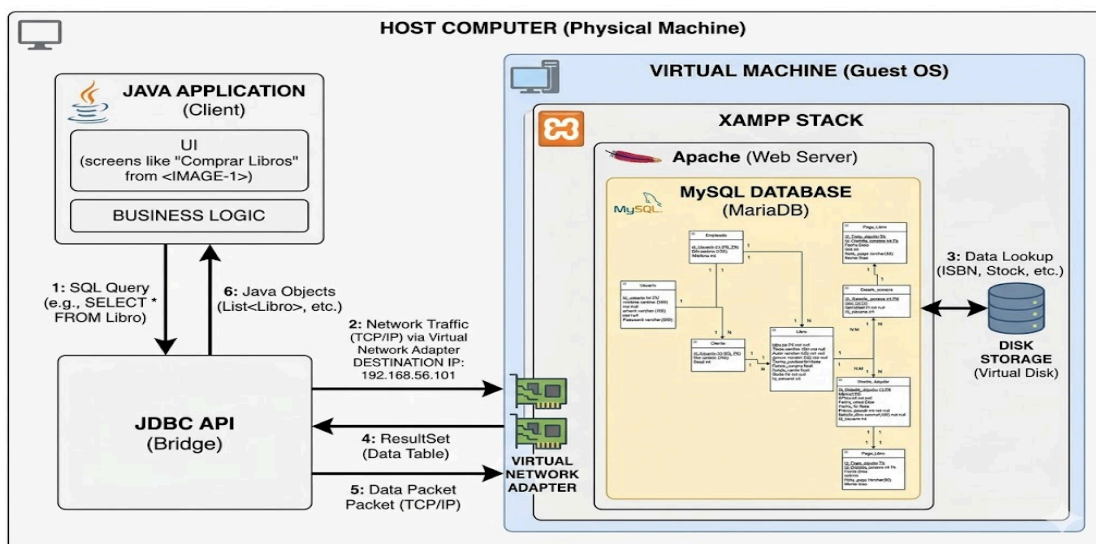
Este bloque se centra en la correcta traducción y redacción de los componentes del proyecto que requieren el uso del idioma inglés.

- **Tecnología empleada:** La mayor parte de los textos se redactaron aprovechando el nivel lingüístico del propio equipo de trabajo. Para

verificar que la ortografía fuera impecable y para traducir términos técnicos muy específicos, recurrimos a un traductor inteligente, lo cual nos sirvió además para incorporar nuevas palabras en su contexto real.

Definición de tecnologías implementadas y su entorno:

Considerando que se necesita que los programas sean compatibles, mientras hay coherencia, uso adecuado del lenguaje, el sistema y el entorno, donde estos hablen el mismo idioma para que funcione correctamente. Para ello se implementó lenguaje de programación Java , mediante el uso del programa NetBeans, en relación con una base de datos específica mediante el lenguaje MySQL, la cual nos permite obtener diagramas y visualizar el orden de los datos y sus bases, garantizando que los datos obtenidos son los correctos.



Fuente: Geminis 2026.

Fase IV: Materialización

Para la realización del programa hemos creado una BD dentro de un servidor Windows, con esto, tenemos la ventaja de poder conectarnos desde cualquier equipo de la red. Como puedo mostrar en la siguiente imagen, se conecta correctamente a la BD teniendo en cuenta que se encuentra en otro ordenador (Máquina Virtual.)

The image shows a Windows Server 2025 virtual machine environment. On the left, the phpMyAdmin interface is open, displaying a table named 'libro' with the following data:

id_libro	titulo	isbn	precio	stock	id_editorial	id_usuario
1	Cien años de soledad	9780000000001	20.00	10	1	1
2	El	9780000000002	22.90	8	2	2
3	Toko blues	9780000000003	17.95	12	1	1
4	de la Tierra	9780000000004	24.00	6	2	2
5	nombre de la rosa	9780000000005	21.50	4	3	3

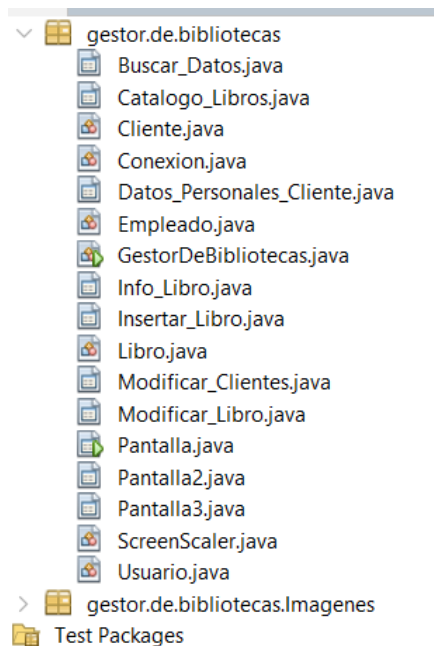
On the right, a Java application window titled 'Gestor_De_Bibliotecas (jfxsa-run)' is shown. It features a login form with fields for 'Email' and 'Contraseña', and a 'Registrarse' button. Below the form, a console window displays the following output:

```

deploy:
jar:
-check-concurrent-runs:
create-temp-run-dir:
[copy] Copying 6 files to C:\Users\df335\Documents\NetBeansProject
jfx-project-run:
[Info] Executing C:\Users\df335\Documents\NetBeansProject\Gestor_De_Bibliotecas (jfxsa-run)
[Info] Conexion correcta
  
```

A red arrow points from the 'Conexion correcta' message in the console to the 'Iniciar Sesión' button in the application window, indicating a successful connection and login attempt.

Todo el hardware creado para el programa es el servidor Windows, el cual necesita de una parte de Software (Código) para funcionar, el cual se muestra a continuación: El software se compone de varias partes (varios JPanel Form, JFrame Form y Java Class), en la siguiente imagen se pueden ver todas las partes fundamentales del programa.



Antes de ver las funciones del programa, se ha de mostrar como conseguir acceder a la BD desde otro ordenador, simplemente, en el método de conectar, en vez de poner el localhost, he puesto la IP del servidor, los cuales cambian, dependiendo el lugar donde se conecte el programa a la red.

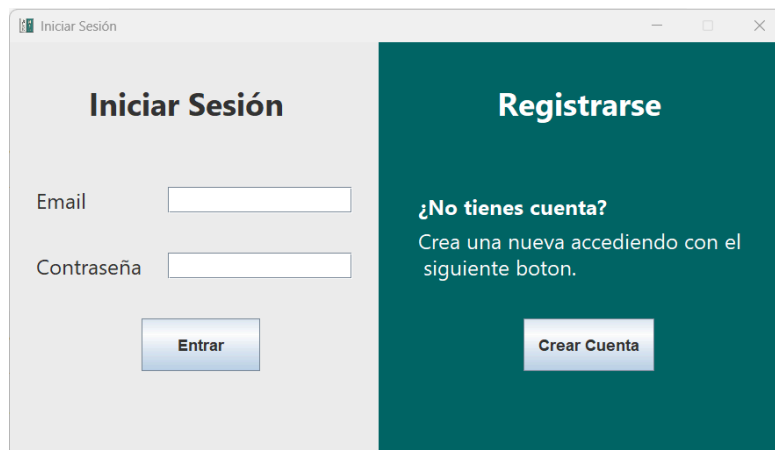

```
package gestor.de.bibliotecas;
import java.sql.*;

public class Conexion {
    static String url = "jdbc:mysql://192.168.1.135:3306/biblioteca";
    static String user = "usuario";
    static String pass = "1234";

    public static Connection conectar(){
        Connection con = null;
        try{
            con = DriverManager.getConnection(url,user,pass);
            System.out.println("Conexion correcta");
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return con;
    }
}
```

Funciones del programa:

-**Pantalla de inicio:** Al acceder, se muestra:



Esta sección permite hacer solo dos cosas, iniciar sesión, o crear cuentas de usuarios. En tal sentido, en caso de querer crear una cuenta, se abre la siguiente pantalla.

-Crear cuenta:

En este JDialog permite ingresar datos para crear una cuenta, los cuales, tienen su validación, es decir, no se puede meter ni un email ni un DNI incorrecto, tampoco se puede dejar nada vacío, ya que el programa enviará un mensaje o excepción por no cumplir con los datos requeridos. Siendo el código que crea la cuenta es el siguiente: Lo único que hace es meter los datos a la BD.

```
public void crearusuario(String nombre , String dni, String correo , String contraseña){
    String nuevo = "INSERT INTO usuario (tipo, nombre_completo, dni, email, contraseña) VALUES ('cliente', ?, ?, ?, ?)";

    try{
        Connection con = Conexion.conectar();
        PreparedStatement ps = con.prepareStatement(nuevo);
        ps.setString(1, nombre);
        ps.setString(2, dni);
        ps.setString(3, correo);
        ps.setString(4, contraseña);
        ps.executeUpdate();
        JOptionPane.showMessageDialog(null, "Cuenta creada correctamente.");
    }
    catch(SQLException ex)
    {
        JOptionPane.showMessageDialog(null, "Error al crear cuenta.");
    }
}
```

Por su parte, en la validación de datos ingresados, se ha asignado este código a un botón.

```
private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    if (jTextField3.getText().isEmpty() ||
        jTextField4.getText().isEmpty() ||
        jTextField5.getText().isEmpty() ||
        jPasswordField2.getPassword().length == 0) {
        JOptionPane.showMessageDialog(null, "Debes rellenar todos los campos.");
    } else {
        String validacionEmail = "[A-Za-z0-9]+@[A-Za-z0-9.-]+\\.([A-Za-z]{2,})$";
        if (Pattern.matches(validacionEmail, jTextField4.getText())) {
            String validacionDNI = "[0-9]{8}[A-Z]$";
            if (Pattern.matches(validacionDNI, jTextField5.getText())) {
                if (jCheckBox1.isSelected()) {
                    Usuario usuario = new Usuario();
                    usuario.crearusuario(jTextField3.getText(), jTextField5.getText(), jTextField4.getText(), new String(jPasswordField2.getPassword()));
                    jTextField3.setText("");
                    jTextField4.setText("");
                    jTextField5.setText("");
                    jPasswordField2.setText("");
                    jCheckBox1.setSelected(false);
                } else {
                    JOptionPane.showMessageDialog(null, "Debes aceptar las políticas de privacidad.");
                }
            } else {
                JOptionPane.showMessageDialog(null, "DNI no válido.");
            }
        } else {
            JOptionPane.showMessageDialog(null, "Correo no válido.");
        }
    }
}
```

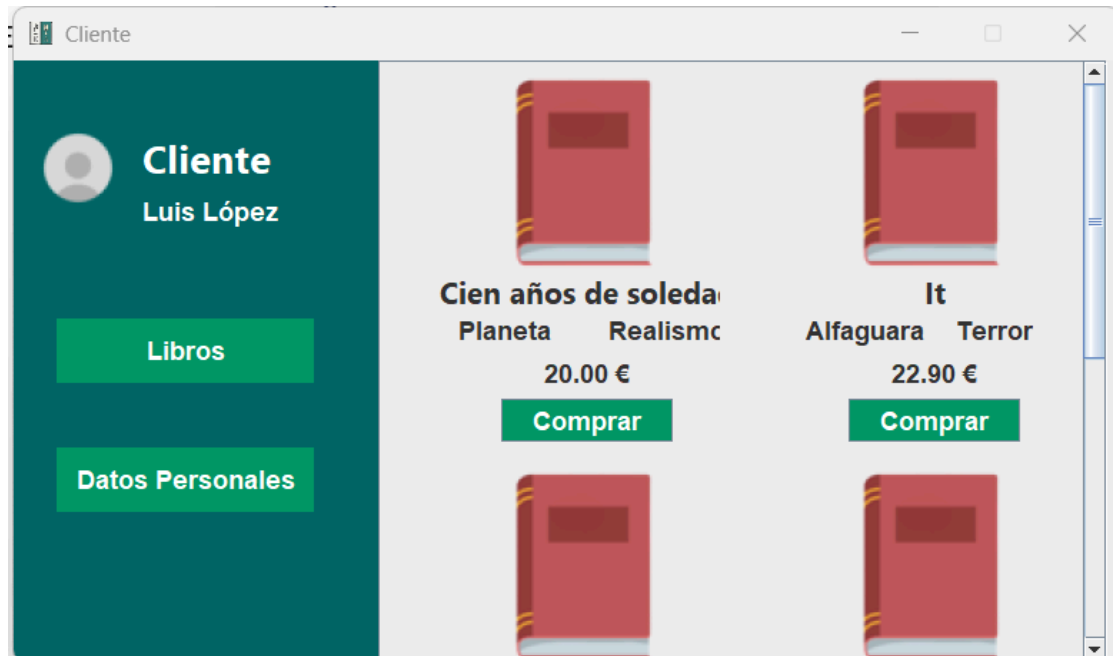
En el mismo orden de ideas, se continúa con la parte de iniciar sesión, una vez introducido los datos, lo único que hace el programa es ver si se encuentran esos datos, y en caso de que si se fija en la parte de tipo y dependiendo de si es empleado o cliente muestra una u otra ventana, si no se encuentran los datos, da error. Dicho código, es el siguiente.

```
public void iniciarsesion(String correo, String contraseña, javax.swing.JFrame pantalla){
    String acceder = "SELECT tipo, nombre_completo FROM usuario WHERE email=? AND contraseña=?";

    try(Connection con = Conexion.conectar();
        PreparedStatement ps = con.prepareStatement(acceder)){
        ps.setString(1, correo);
        ps.setString(2, contraseña);

        ResultSet rs = ps.executeQuery();
        if(rs.next()){
            String tipo = rs.getString("tipo");
            String nombre = rs.getString("nombre_completo");
            if(tipo.equals("cliente")){
                new Pantalla3(nombre, correo).setVisible(true);
                pantalla.dispose();
            } else{
                new Pantalla2(nombre).setVisible(true);
                pantalla.dispose();
            }
        } else {
            JOptionPane.showMessageDialog(null, "Datos incorrectos.");
        }
    } catch (SQLException e){
        JOptionPane.showMessageDialog(null, "Error al iniciar sesion.");
    }
}
```

-Pantalla de cliente:



-Pantalla de empleado:



Las Funciones que tiene el programa, se dividen en las opciones para empleados y aquellas para clientes. Sabiendo esto, cómo se puede ver en la

siguiente imagen, el programa muestra un panel izquierdo con el tipo de usuario (Empleado), también se puede ver que debajo muestra el nombre del usuario, esto va cambiando en función de quién accede.

Asimismo, el panel izquierdo, tiene 4 botones, según los pulsas, muestra en la derecha una u otra cosa, la parte derecha es un JPanel, a este se le da un diseño agregándole un JPanel Form, ese código se encuentra en los botones, se puede ver en la siguiente imagen.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    java.awt.CardLayout card = (java.awt.CardLayout)jPanel1.getLayout();
    card.show(jPanel1, "insertar");
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {
    java.awt.CardLayout card = (java.awt.CardLayout)jPanel1.getLayout();
    card.show(jPanel1, "modificar");
    Libro libro = new Libro();
    libro.CargarLibros(p2.getComboBox());
}

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {
    java.awt.CardLayout card = (java.awt.CardLayout)jPanel1.getLayout();
    card.show(jPanel1, "clientes");
    Empleado empleado = new Empleado();
    empleado.cargar_clientes(p3.getComboBox());
}

private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    java.awt.CardLayout card = (java.awt.CardLayout)jPanel1.getLayout();
    card.show(jPanel1, "buscar");

    Empleado empleado = new Empleado();
    empleado.Mostrar_en_Tabla(p4.getComboBox(), p4.getTable());
}
```

Siendo así cómo se asignó la primera funcionalidad de empleado, insertar libro, la pantalla de esta función es la siguiente.

The screenshot shows a Java Swing window titled "Empleado". On the left, there is a dark green sidebar with a user profile icon and the name "Ana López". Below the name are four buttons: "Insertar Libro", "Modificar Libro", "Modificar Clientes", and "Buscar Datos". The main area of the window is light gray and contains a form for adding a book. The form has the following fields and buttons:

- Título:** A text input field.
- Precio:** A text input field followed by a Euro symbol (€).
- ISBN:** A text input field.
- Stock:** A text input field.
- Categ:** A text input field.
- Edito:** A text input field.
- Añadir Libro:** A green button at the bottom center.
- Limpiar Campos:** A red button at the bottom right.

Al pulsar el botón de limpiar campos, simplemente se borran todos los `textField`, y cuando pulso el de añadir libro, primero busca a ver si hay una editorial y una categoría con el mismo nombre, si la hay obtiene su ID, si no, lo crea y obtiene el ID de los creados, luego, con esos ID hace la creación del libro. El código es el siguiente.

```
public void AgregarLibro() {  
  
    Connection con = Conexion.conectar();  
    //INSERTAR EDITORIAL  
    try{  
        String insertar_editorial = "INSERT INTO editorial(nombre) VALUES(?)";  
        PreparedStatement ps = con.prepareStatement(insertar_editorial);  
        ps.setString(1, this.editorial);  
        ps.executeUpdate();  
    }catch(Exception e){  
        e.printStackTrace();  
    }  
  
    //OBTENER ID DE LA EDITORIAL  
    try{  
  
        String obtener_id_editorial = "SELECT id_editorial FROM editorial WHERE nombre = ?";  
        PreparedStatement ps = con.prepareStatement(obtener_id_editorial);  
        ps.setString(1, this.editorial);  
  
        ResultSet rs = ps.executeQuery();  
        if(rs.next()){  
            this.id_editorial = rs.getInt(1);  
        }  
  
    }catch(Exception e){  
        e.printStackTrace();  
    }  
  
    //INSERTAR CATEGORIA  
    try{  
        String insertar_categoria = "INSERT INTO categoria(nombre) VALUES(?)";  
        PreparedStatement ps = con.prepareStatement(insertar_categoria);  
        ps.setString(1, this.categoria);  
        ps.executeUpdate();  
    }catch(Exception e){
```

```

        e.printStackTrace();
    }

    //OBTENER ID DE LA CATEGORIA
    try{

        String obtener_id_categoria = "SELECT id_categoria FROM categoria WHERE nombre = ?";
        PreparedStatement ps = con.prepareStatement(obtener_id_categoria);
        ps.setString(1, this.categoria);
        ResultSet rs = ps.executeQuery();
        if(rs.next()){
            this.id_categoria = rs.getInt(1);
        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    //INSERTAR LIBRO
    String insertar = "INSERT INTO libro(titulo,isbn,precio,stock,id_editorial,id_categoria) VALUES (?, ?, ?, ?, ?, ?)";
    try{
        PreparedStatement ps = con.prepareStatement(insertar);
        ps.setString(1, this.titulo);
        ps.setString(2, this.isbn);
        ps.setDouble(3, this.precio);
        ps.setInt(4, this.stock);
        ps.setInt(5, this.id_editorial);
        ps.setInt(6, this.id_categoria);
        ps.executeUpdate();
        JOptionPane.showMessageDialog(null, "Libro añadido correctamente.");
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, "Error al agregar libro.");
        e.printStackTrace();
    }
}

```

Ahora bien, la función de modificar el libro, la pantalla es la siguiente:.

El botón filtrar, lo único que hace es un select con el nombre o ISBN del libro, todos los resultados obtenidos, los convierte en objetos y los añade a la lista, el objeto lo crea con el ISBN y el nombre del libro. El código es el siguiente:

```
public void FiltroLibros(JComboBox lista, JTextField filtro){

    String busqueda = filtro.getText();
    Connection con = Conexion.conectar();

    try{

        lista.removeAllItems();

        if(busqueda.equals("")){
            lista.addItem("                -- Inserte un Libro --");
        }

        String sql = "SELECT isbn, titulo FROM libro WHERE titulo LIKE ?";
        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, "%" + busqueda + "%");

        ResultSet rs = ps.executeQuery();
        String isbn;
        String titulo;
        while(rs.next()){
            isbn = rs.getString("isbn");
            titulo = rs.getString("titulo");
            String resultado = isbn + " | " + titulo;
            lista.addItem(resultado);
        }

    }catch(Exception e){
        e.printStackTrace();
    }

}
```

Ahora el combobox, este tiene un método que cuando seleccionas una opción hace un select de todos los datos del libro, y obtiene el Id de categoría y editorial y luego hace otro select dentro de sus tablas para obtener el nombre, luego los datos obtenidos, los mete en el JTextField. Ante lo mencionado, se expone el código:


```

public void MostrarDatos(JComboBox lista, JTextField titulo, JTextField precio, JTextField stock, JTextField categoria, JTextField editorial)
{
    if(lista.getSelectedItem() == null){
        return;
    }

    String seleccion = lista.getSelectedItem().toString();
    String[] dato= seleccion.split("\\|");

    try{

        String sql = "SELECT * FROM libro WHERE isbn = ?";

        Connection con = Conexion.conectar();

        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, dato[0]);

        ResultSet rs = ps.executeQuery();
        if(rs.next()){

            //OBTENGO TODOS LOS DATOS
            String tit = rs.getString("titulo");
            double prec = rs.getDouble("precio");
            int sto = rs.getInt("stock");
            int id_cat = rs.getInt("id_categoria");
            int id_edit = rs.getInt("id_editorial");

```

```

// OBTENER E INTRODUCIR EN LOS CAMPOS DE TEXTO LOS NOMBRES DE CATEGORIA Y EDITORIAL A PARTIR DE SU ID

// SACO EL NOMBRE DE CATEGORIA PARA PODER METERLOS EN LOS CAMPOS DE TEXTO
String obtener_id_categoria = "SELECT nombre FROM categoria WHERE id_categoria = ?";
PreparedStatement ps1 = con.prepareStatement(obtener_id_categoria);
ps1.setInt(1, id_cat);
ResultSet rs1 = ps1.executeQuery();
if(rs1.next()){
    String cat = rs1.getString("nombre");
    //ASIGNO EL VALOR DE CATEGORÍA EN EL CAMPO CORRESPONDIENTE
    categoria.setText(cat);
}

// SACO EL NOMBRE DE EDITORIAL PARA PODER METERLOS EN LOS CAMPOS DE TEXTO
String obtener_id_editorial = "SELECT nombre FROM editorial WHERE id_editorial = ?";
PreparedStatement ps2 = con.prepareStatement(obtener_id_editorial);
ps2.setInt(1, id_edit);
ResultSet rs2 = ps2.executeQuery();
if(rs2.next()){
    String cat = rs2.getString("nombre");
    //ASIGNO EL VALOR DE LA EDITORIAL EN EL CAMPO CORRESPONDIENTE
    editorial.setText(cat);
}

//ASIGNO LOS DEMAS VALORES A LOS CAMPOS DE TEXTO PARA QUE EL USUARIO LOS VEA Y MODIFIQUE
titulo.setText(tit);
precio.setText(String.valueOf(prec));
stock.setText(String.valueOf(sto));
}
else{
    //SI NO SE ENCUENTRA EL LIBRO EN LA BD, SE DEJA TODO EN BLANCO
    titulo.setText("");
    precio.setText(String.valueOf(""));
    stock.setText(String.valueOf(""));
    categoria.setText("");
    editorial.setText("");
}

```

Posteriormente, se desarrolló la sección modificar libro, este boton hace algo similar al de insertar libro, pero con un update, no adjunto el

código porque es lo mismo. Finalmente, se presenta el botón eliminar libro es un simple delete, pero borra solo el libro, no la editorial ni la categoría, porque varios libros pueden tener la misma.

```
public void EliminarLibro(JComboBox lista){  
  
    Connection con = Conexion.conectar();  
    String seleccion = lista.getSelectedItem().toString();  
  
    String [] dato = seleccion.split("\\|");  
    String isbn = dato[0];  
  
    try{  
  
        String sql = "DELETE FROM libro WHERE isbn = ?";  
        PreparedStatement ps = con.prepareStatement(sql);  
        ps.setString(1, isbn);  
        ps.executeUpdate();  
        JOptionPane.showMessageDialog(null, "Libro eliminado correctamente.");  
  
    }catch(Exception e){  
        JOptionPane.showMessageDialog(null, "Error al eliminar libro.");  
        e.printStackTrace();  
    }  
  
}
```

Ahora bien, se tiene el modificar clientes, representado con:

Esta parte no la voy a explicar porque es exactamente igual que modificar libro, pero con clientes, lo único diferente es que solo muestra los usuarios que en la BD, en la columna tipo tienen “cliente”. Por último, se llega a la parte de buscar datos, la cual muestra lo siguiente.



Empleado
Ana López

Insertar Libro

Modificar Libro

Modificar Clientes

Buscar Datos

Tabla: Usuarios

Buscar: **Filtrar**

Id	Tipo	Nombre	DNI	Email	Telefono
1	emplea...	Ana Ló...	123456...	ana@m...	
2	emplea...	Carlos ...	234567...	carlos...	
3	cliente	Sofía T...	345678...	sofia@...	123456...
4	cliente	Pablo ...	456789...	pablo@...	
5	cliente	Elena ...	567890...	elena@...	
7	cliente	Luis Ló...	321654...	luis@m...	123456...

En esta parte, la función de la tabla que eliges, se va actualizando la tabla de abajo, la clase que va dentro del combobox es la siguiente.

```
public void Mostrar_en_Tabla(JComboBox tablas, JTable tabla){  
  
    Connection con = Conexion.conectar();  
    String seleccion = tablas.getSelectedItem().toString();  
  
    DefaultTableModel modelo = new DefaultTableModel();  
  
    if (seleccion.equals("Usuarios")){  
  
        try{  
  
            modelo.setRowCount(0);  
            modelo.setColumnCount(0);  
  
            modelo.addColumn("Id");  
            modelo.addColumn("Tipo");  
            modelo.addColumn("Nombre");  
            modelo.addColumn("DNI");  
            modelo.addColumn("Email");  
            modelo.addColumn("Telefono");  
  
            String sql = "SELECT id_usuario, tipo, nombre_completo, dni, email, telefono FROM usuario";  
  
            PreparedStatement ps = con.prepareStatement(sql);  
            ResultSet rs = ps.executeQuery();  
  
            while(rs.next()){  
                Object[] fila = new Object[6];  
                fila[0] = rs.getString("id_usuario");  
                fila[1] = rs.getString("tipo");  
                fila[2] = rs.getString("nombre_completo");  
                fila[3] = rs.getString("dni");  
                fila[4] = rs.getString("email");  
                fila[5] = rs.getString("telefono");  
                modelo.addRow(fila);  
            }  
        }  
    }  
}
```

Centro De Bibliotecas (ifvca.rim)

running

```
        tabla.setModel(modelo);

    } catch (Exception e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(null, "No se puede cargar la tabla de usuarios.");
    }
}

if (seleccion.equals("Libros")) {

    try {

        modelo.setRowCount(0);
        modelo.setColumnCount(0);

        modelo.addColumn("Id");
        modelo.addColumn("Título");
        modelo.addColumn("ISBN");
        modelo.addColumn("Precio");
        modelo.addColumn("Stock");
        modelo.addColumn("ID_Edi");
        modelo.addColumn("ID_Cat");

        String sql = "SELECT * FROM libro";

        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            Object[] fila = new Object[7];
            fila[0] = rs.getString("id_libro");
            fila[1] = rs.getString("titulo");
            fila[2] = rs.getString("isbn");
            fila[3] = rs.getString("precio");
            fila[4] = rs.getString("stock");
            fila[5] = rs.getString("id_editorial");
            fila[6] = rs.getString("id_categoria");
```

```

        fila[6] = rs.getString("id_categoria");
        modelo.addRow(fila);
    }

    tabla.setModel(modelo);

} catch (Exception e) {
    e.printStackTrace();
    JOptionPane.showMessageDialog(null, "No se puede cargar la tabla de libros.");
}

}

if (seleccion.equals("Editoriales")) {

    try {

        modelo.setRowCount(0);
        modelo.setColumnCount(0);

        modelo.addColumn("Id");
        modelo.addColumn("Nombre");

        String sql = "SELECT * FROM editorial";
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            Object[] fila = new Object[2];
            fila[0] = rs.getString("id_editorial");
            fila[1] = rs.getString("nombre");
            modelo.addRow(fila);
        }

        tabla.setModel(modelo);

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

El filtro de esta parte, es simplemente un select con los datos introducidos y borra toda la tabla, dejando solo lo que se encuentra en el select. El código es el siguiente. Muestro solo el de usuario porque los demás son iguales, adaptados a su tabla.

```
public void Filtrar_Tablas(JComboBox combobox, JTable tabla, JTextField filtro){

    DefaultTableModel modelo = (DefaultTableModel) tabla.getModel();
    modelo.setRowCount(0);

    String busqueda = filtro.getText();
    Connection con = Conexion.conectar();
    String seleccion = combobox.getSelectedItem().toString();

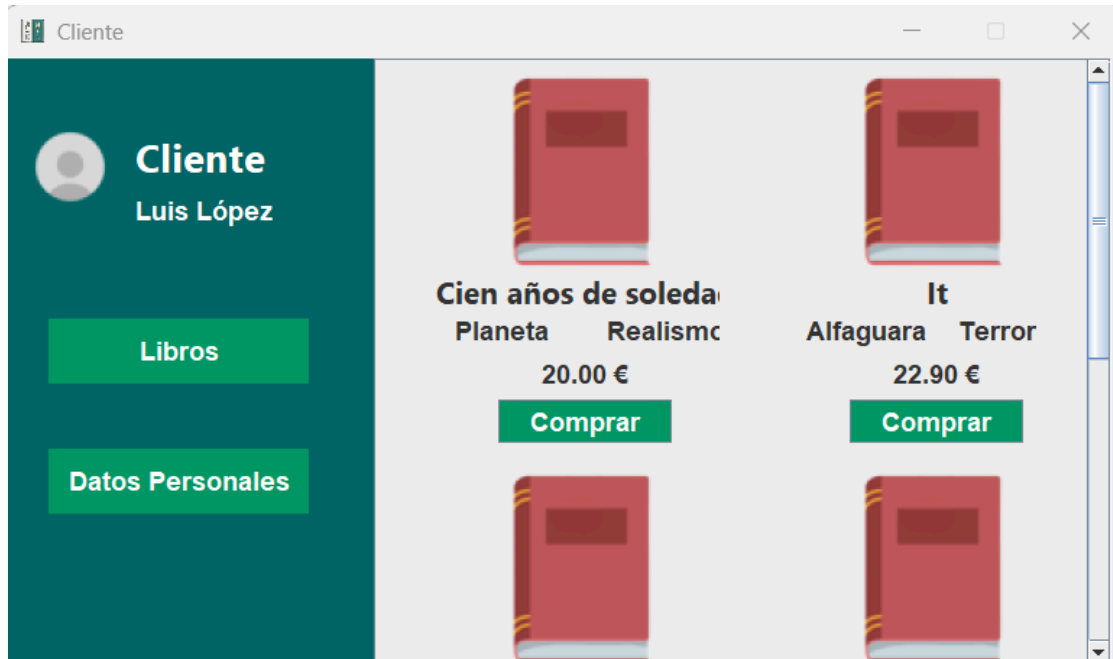
    if(busqueda.equals("")){
        Mostrar_en_Tabla(combobox, tabla);
        return;
    }

    try {

        if (seleccion.equals("Usuarios")) {
            String sql = "SELECT * FROM usuario WHERE nombre_completo LIKE ? OR dni LIKE ?";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString(1, "%" + busqueda + "%");
            ps.setString(2, "%" + busqueda + "%");
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                Object[] fila = new Object[6];
                fila[0] = rs.getString("id_usuario");
                fila[1] = rs.getString("tipo");
                fila[2] = rs.getString("nombre_completo");
                fila[3] = rs.getString("dni");
                fila[4] = rs.getString("email");
                fila[5] = rs.getString("telefono");
                modelo.addRow(fila);
            }
        }
    }
```

En consecuencia, se obtiene la pantalla que muestra la sección de empleados, y que puede modificar al cliente, el primer apartado a encontrar es el de libros, este se ve así.



Esta pantalla tiene en la parte derecha un JPanel, que contiene un JScrollPane con otro JPanel, el scrollpanel permite poder tener muchos libros en plan catálogo. Mientras el JPanel del scrollpanel tiene un layout configurado para poder meter el contenido de tal manera que esté así, para que este panel muestre todos los libros, básicamente va recibiendo los JPanel Form que se van creando por cada libro, porque cada libro es un JPanel Form, esto lo hace un método que he introducido en el botón de libros, también está en el inicio de la pantalla (panel derecho), el código de dicho método es el siguiente.


```
public void Catalogo_Libros (javax.swing.JPanel Catalogo_Libros, java.awt.Image imgEscalada){

    Connection con = Conexion.conectar();

    Catalogo_Libros.removeAll();

    try{

        String sql = "SELECT titulo, precio, id_editorial, id_categoria from libro";
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        while(rs.next()){

            String id_editorial = rs.getString("id_editorial");
            String id_categoria = rs.getString("id_categoria");

            String Obtener_Editorial = "SELECT nombre FROM editorial WHERE id_editorial = ?";
            PreparedStatement ps1 = con.prepareStatement(Obtener_Editorial);
            ps1.setString(1, id_editorial);
            ResultSet rs1 = ps1.executeQuery();

            String editorial = "";

            if(rs1.next()){
                editorial = rs1.getString("nombre");
            }

            String Obtener_Categoria = "SELECT nombre FROM categoria WHERE id_categoria = ?";
            PreparedStatement ps2 = con.prepareStatement(Obtener_Categoria);
            ps2.setString(1, id_categoria);
            ResultSet rs2 = ps2.executeQuery();

            String categoria = "";

            if(rs2.next()){
                categoria = rs2.getString("nombre");
            }

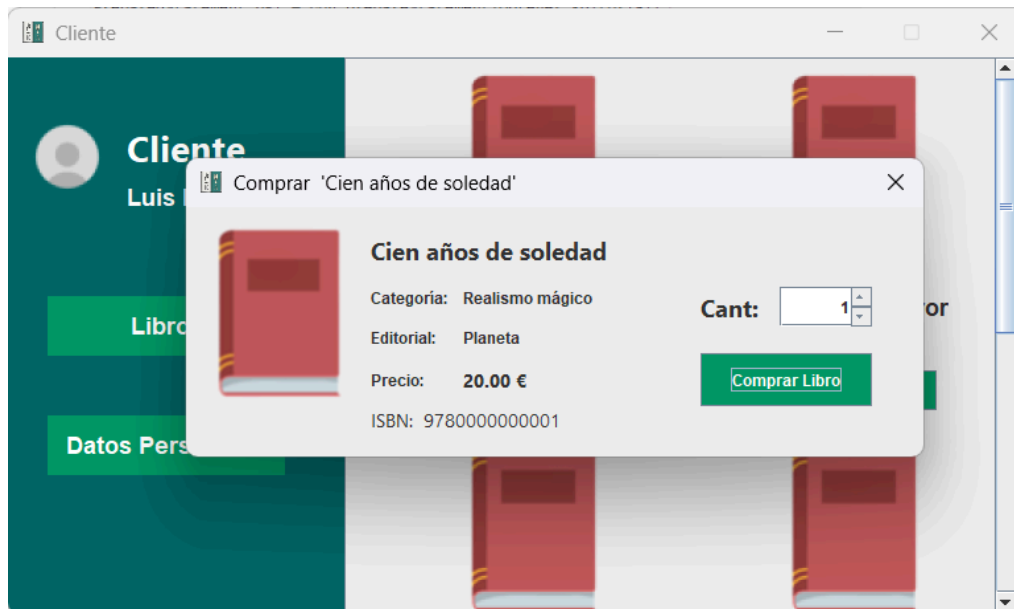
            String nombre = rs.getString("titulo");
            String precio = rs.getString("precio");
            precio = (precio + " €");

            Info_Libro libro = new Info_Libro(nombre, editorial, categoria, precio, imgEscalada);
            Catalogo_Libros.add(libro);
        }

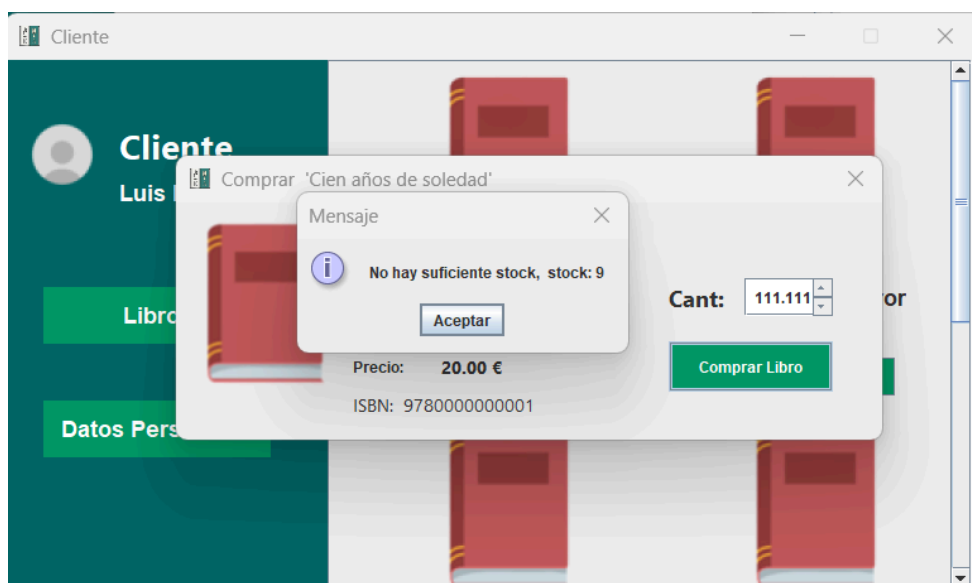
        Catalogo_Libros.revalidate();
        Catalogo_Libros.repaint();

    }catch(Exception e){
```

En caso de darle a comprar a un libro, se abre un jDialog, este muestra lo siguiente:



Como se puede ver, los datos del libro permiten seleccionar una cantidad específica de libros y poder comprarlos. Por ello, los datos los obtengo con un select, y la parte importante es la de comprar, ya que al hacer la compra, en caso de que no haya stock suficiente, muestra una pantalla como esta.



En caso del stock, resta la cantidad seleccionada al stock de la BD, cada vez que un cliente compre un libro. Todo el código de comprar libro es el siguiente.

```
public void Comprar_Libro(JSpinner cantidad, JLabel isbn){  
    Connection con = Conexion.conectar();  
  
    try{  
        String sql = "SELECT stock FROM libro WHERE isbn = ?";  
        PreparedStatement ps = con.prepareStatement(sql);  
        ps.setString(1, isbn.getText());  
        ResultSet rs = ps.executeQuery();  
        int stock = 0;  
        if(rs.next()){  
            stock = rs.getInt("stock");  
        }  
  
        int cant = (Integer) cantidad.getValue();  
  
        if(cant <= stock){  
            String cambiar_stock = "UPDATE libro SET stock = stock - ? WHERE isbn = ?";  
            PreparedStatement ps1 = con.prepareStatement(cambiar_stock);  
            ps1.setInt(1, cant);  
            ps1.setString(2, isbn.getText());  
            ps1.executeUpdate();  
            JOptionPane.showMessageDialog(null, "Compra realizada correctamente.");  
        } else {  
            JOptionPane.showMessageDialog(null, "No hay suficiente stock, stock: " + stock);  
        }  
    } catch (Exception e){  
        JOptionPane.showMessageDialog(null, "Error al comprar libro.");  
        e.printStackTrace();  
    }  
}
```

Para concluir el desarrollo del programa se muestra la sección de cliente con sus datos personales, de los cuales no se puede modificar el DNI porque este es único e irrepitable.

The screenshot displays a web application window titled "Cliente". On the left, a dark teal sidebar contains a profile card for "Luis López" with a placeholder icon, and two green buttons labeled "Libros" and "Datos Personales". The main content area is light gray and contains a form with the following fields:

- Nombre:
- Email:
- Teléfono:
- DNI:
- Contraseña:

Below the password field is a green button labeled "Modificar Datos".

```

public void Modificar_Datos_Personales (JTextField nombre, JTextField dni, JTextField email, JTextField telefono, JPasswordField contraseña){

    Connection con = Conexion.conectar();
    String sql = "UPDATE usuario SET nombre_completo = ?, email = ?, telefono = ?, contraseña = ? WHERE dni = ?";

    String pass = new String(contraseña.getPassword());

    String validacion_telefono = "^([0-9]{9})$";

    if(!Pattern.matches(validacion_telefono, telefono.getText()) & !telefono.getText().equals("")){
        JOptionPane.showMessageDialog(null, "El teléfono no es válido.");
        return;
    }

    String validacion_correo = "^([A-Za-z0-9]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,})$";

    if(!Pattern.matches(validacion_correo, email.getText())){
        JOptionPane.showMessageDialog(null, "Correo no válido.");
        return;
    }

    try{

        PreparedStatement ps = con.prepareStatement(sql);
        ps.setString(1, nombre.getText());
        ps.setString(2, email.getText());
        ps.setString(3, telefono.getText());
        ps.setString(4, pass);
        ps.setString(5, dni.getText());
        ps.executeUpdate();

        JOptionPane.showMessageDialog(null, "Datos actualizados correctamente.");

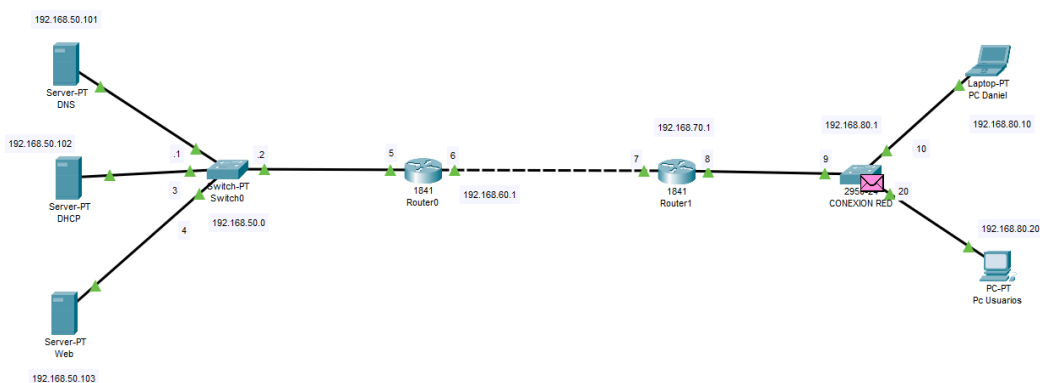
    }catch(Exception e){
        e.printStackTrace();
        JOptionPane.showMessageDialog(null, "Error al modificar datos.");
    }

}

```

La red de BookFlow en Cisco Packet Tracer

Para mostrar cómo funciona BookFlow en un entorno real, el equipo ha simulado toda su infraestructura de red usando **Cisco Packet Tracer**. Esta simulación representa exactamente lo que ocurriría si el sistema estuviera desplegado en una empresa u organización real.



En ella se pueden ver los **tres servidores** que dan vida a la aplicación (web, base de datos y red), los **equipos de los usuarios** —tanto clientes como empleados— y los dispositivos de red que los conectan entre sí. Cuando un usuario abre BookFlow, su petición viaja por esta red hasta llegar al servidor correspondiente, que le devuelve la información: un catálogo de libros, una compra procesada, los datos de su cuenta...



La simulación demuestra que todos los componentes del sistema se comunican correctamente, tal y como lo harían en un despliegue real, confirmando que BookFlow está diseñado para funcionar en red de forma estable y fiable.

Conclusión:

El desarrollo de BookFlow ha demostrado la viabilidad de integrar múltiples disciplinas tecnológicas para resolver un problema real y cotidiano como es la administración de una biblioteca. A través de este programa, el equipo no solo ha logrado diseñar un software funcional y un almacén de datos seguro, sino también una plataforma web completamente adaptada a las necesidades del usuario moderno. En definitiva, BookFlow se consolida como una solución integral, eficiente e intuitiva que optimiza el control de los libros y simplifica la experiencia digital de los lectores.

Referencias de plataformas utilizadas

Conexión MySQL con Java

<https://www.youtube.com/watch?v=mhDqL2SUXJc>

Insertar datos a la Base de Datos desde el programa de Java

<https://www.youtube.com/watch?v=oalqlZnnvs8&t=452s>